

```

--
=====
=====
-- SECTION: VIEWS (public schema) -- 4 views, all
diagnostic/introspection
-- No materialized views, no standalone sequences
(all PKs use gen_random_uuid()).
--
=====
=====

CREATE OR REPLACE VIEW
public.payment_reconciliation_summary AS
SELECT p.id AS payment_id,
       p.order_id,
       p.amount AS payment_amount,
       o.total_amount AS order_total_amount,
       p.status AS payment_status,
       o.status AS order_status,
       (p.amount - o.total_amount) AS
reconciliation_drift
FROM (payments p
      LEFT JOIN orders o ON ((p.order_id = o.id)));

CREATE OR REPLACE VIEW
public.rls_enforcement_status AS
SELECT tablename,
       rowsecurity AS rls_enabled
FROM pg_tables
WHERE (schemaname = 'public'::name)
ORDER BY tablename;

CREATE OR REPLACE VIEW public.schema_contract AS
SELECT table_schema,
       table_name,
       column_name,

```

```

        data_type,
        is_nullable,
        column_default
    FROM information_schema.columns
    WHERE ((table_schema)::name = 'public'::name)
    ORDER BY table_name, ordinal_position;

CREATE OR REPLACE VIEW
public.security_advisory_summary AS
SELECT routine_name,
       routine_type,
       security_type
    FROM information_schema.routines
    WHERE (((routine_schema)::name = 'public'::name)
    AND ((security_type)::text = 'DEFINER'::text));

--
=====
=====
-- SECTION: SCHEDULED JOBS (pg_cron) -- 4 active
jobs
-- Requires pg_cron extension (already in
01_schemas_extensions.sql).
-- Apply LAST, after all functions exist.
--
=====
=====

SELECT cron.schedule('hg_promo_auto_deactivate', '0
* * * *',
    $$UPDATE public.promo_codes SET is_active=FALSE
WHERE is_active=TRUE AND ends_at IS NOT NULL AND
ends_at < NOW()$$);

SELECT cron.schedule('hg_tier_maintenance_daily',
'0 2 * * *',
    $$SELECT
public.hg_run_tier_maintenance_check(NULL)$$);

SELECT cron.schedule('hg_abandoned_cart_recovery',

```

```

    '**/30 * * * *',
    $$UPDATE public.abandoned_carts SET
    last_recovery_sent_at=NOW(),recovery_attempts=recovery_attempts+1,next_recovery_at=NOW()+INTERVAL '2
    hours' WHERE is_recovered=FALSE AND
    (next_recovery_at IS NULL OR
    next_recovery_at<=NOW()) AND recovery_attempts<3
    AND created_at<NOW()-INTERVAL '1 hour'$$);

SELECT cron.schedule('hg_hp_expiry_daily', '0 1 * *
    *',
    $$SELECT public.hg_run_hp_expiry_check(180)$$);

-- NOTE: No storage buckets exist on this project
-- ("test 2"), so there is
-- nothing to migrate for storage.buckets / storage
-- policies. If your
-- Production Backend project uses Storage for
-- images (menu items, profile
-- photos, banners), that must be captured
-- separately from that project.

```